

Model Serialization Lab (Practical)

Welcome to the practical lab for Model Serialization! In this session, you will take your trained machine learning pipeline (which handles data cleaning, encoding, scaling, and the model itself), serialize it to disk using **Joblib**, and verify that it can be reloaded for instant inference. This bridges your machine learning workflow directly to your FastAPI backend!

1. Concept Overview

In this lab, you will:

- Train a Scikit-Learn **Pipeline** that combines `SimpleImputer`, `StandardScaler`, `OneHotEncoder`, and a `LinearRegression` model to predict annual rainfall.
- Use **Joblib** to save the entire pipeline into a single `.joblib` file. (A massive advantage of pipelines is that you only need to save one file, rather than keeping track of separate scalers and models!)
- Write a verification script that loads the saved pipeline from disk and makes a prediction on fresh input data.

2. Step-by-Step Lab Instructions

Step 1: Create the Export Script (`export_model.py`)

Ensure your `rainfall.csv` file is in your project folder. Create a new file named `export_model.py` and add the following code:

```
# export_model.py
import pandas as pd
import joblib
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
```

```
# 1. Load the Rainfall Dataset
```

```

print("Loading data and training model...")
df = pd.read_csv("rainfall.csv")
df_clean = df.dropna(subset=['ANNUAL']).copy()

features = ['SUBDIVISION', 'JAN', 'FEB', 'MAR', 'APR', 'MAY']
target = 'ANNUAL'

X = df_clean[features]
y = df_clean[target]

# 2. Split data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

# 3. Build Preprocessing Steps
numeric_features = ['JAN', 'FEB', 'MAR', 'APR', 'MAY']
categorical_features = ['SUBDIVISION']

numeric_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='mean')),
    ('scaler', StandardScaler())
])

categorical_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='most_frequent')),
    ('onehot', OneHotEncoder(handle_unknown='ignore',
sparse_output=False))
])

preprocessor = ColumnTransformer(
    transformers=[
        ('num', numeric_transformer, numeric_features),
        ('cat', categorical_transformer, categorical_features)
    ])

# 4. Create and Train the Final Pipeline
model_pipeline = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('regressor', LinearRegression())
])

model_pipeline.fit(X_train, y_train)

# 5. Serialize and Save the Entire Pipeline using Joblib
joblib.dump(model_pipeline, "rainfall_model_pipeline.joblib")

print("Pipeline successfully exported to disk as
'rainfall_model_pipeline.joblib'!")

```

Step 2: Create a Verification Script (test_loaded_model.py)

Create a second file named `test_loaded_model.py` to simulate how your FastAPI backend will load and use this file for live inference:

```
# test_loaded_model.py
import pandas as pd
import joblib

# 1. Load the pipeline from disk (Done once at server startup)
# This single object contains your model, scalers, and encoders!
loaded_pipeline = joblib.load("rainfall_model_pipeline.joblib")

# 2. Simulate incoming raw user input from a frontend
# Note: Since the pipeline expects column names, we pass a DataFrame
new_input = pd.DataFrame({
    'SUBDIVISION': ['Kerala'],
    'JAN': [28.5],
    'FEB': [14.2],
    'MAR': [33.1],
    'APR': [110.4],
    'MAY': [250.8]
})

# 3. Make prediction
# The pipeline automatically imputes, encodes, scales, and predicts in one
# step
predicted_rainfall = loaded_pipeline.predict(new_input)

print("=== Live Inference Test ===")
print(f"Input Region: Kerala | Early Year Rain: ~437 mm")
print(f"Predicted Annual Rainfall: {predicted_rainfall[0]:.2f} mm")
```

Step 3: Run Your Scripts

Open your terminal, make sure your virtual environment is active, and execute the export script followed by the test script:

```
python export_model.py
python test_loaded_model.py
```

Check your project folder—you will see `rainfall_model_pipeline.joblib` ready to be integrated directly into your FastAPI backend (`main.py`) in the next module!

Congratulations on completing **Module 3 (Machine Learning Basics)**!

3. Reference Links & Further Reading

- **Joblib Dump and Load Documentation:** Official guide on object serialization parameters.

- **Scikit-Learn Pipeline Persistence:** Best practices for saving preprocessors and models together.
-